HARDWARE DESIGN · DESIGN SPECIFICATION

Custom USB-C Embedded Development Board

An ESP32-S3 development board powered over USB-C, with Li-ion battery charging, automatic switch-over between USB and battery, an efficient 3.3 V buck regulator, input and ESD protection, and SPI / I²C / UART / GPIO breakouts. Programming and debugging use the chip's own USB port, so no USB-to-serial adapter is needed. This document sets out the requirements, component choices, power budget, pin plan, PCB rules and the bring-up and test plan.

USB-C power + data · 5.1 kΩ CC resistors (power sink)	**2 A** 3.3 V buck regulator (≥ 2.5× the peak load)	**250 mA** Li-ion charge current (one resistor sets it)
0 USB-to-serial chips: native USB-Serial/JTAG	**4** interface groups: SPI · I²C · UART · GPIO	**8 steps** bring-up plan, each with a pass criterion

Status	**Design stage:** architecture, component choices, power budget and test plan complete (this document) · schematic capture and PCB layout next · **not yet fabricated**
Author	Shree Vaishnavi M G
Microcontroller	ESP32-S3-WROOM-1-N4R8 (4 MB flash, 8 MB octal PSRAM, Wi-Fi + BLE 5)
Planned tools	KiCad (schematic + 2-layer PCB) · ESP-IDF (bring-up firmware) · bench supply, multimeter, oscilloscope
Why build it	A small, battery-capable reference board that applies the power and USB-C lessons from my ESP32-S3 product work

1 Requirements

The board should be a general-purpose ESP32-S3 platform that runs from a USB-C cable or a single Li-ion cell, charges that cell safely, survives the abuse a dev board normally gets, and exposes every common interface on standard headers.

ID	Requirement	How the design meets it
R1	Powered and programmed from one USB-C cable, from any USB-C or USB-A-to-C source	USB-C receptacle with 5.1 kΩ resistors on CC1/CC2; ESP32-S3 native USB on GPIO19/20
R2	Program and debug without an extra adapter	Built-in USB-Serial/JTAG: flashing, console and OpenOCD over the same cable
R3	Run from a 1-cell Li-ion/LiPo battery and charge it from USB	MCP73831 charger + automatic USB/battery switch-over
R4	Stable 3.3 V during Wi-Fi transmit peaks, at any input from 3.0 V (battery) to 5.25 V (USB)	TLV62569 buck: 2.5–5.5 V input, 2 A, 100% duty mode at low battery
R5	Survive ESD on the connector, an overcurrent fault and voltage spikes	ESD array on D+/D−, TVS diode on VBUS, PTC resettable fuse
R6	SPI, I²C, UART and GPIO on 2.54 mm headers, plus one plug-in I²C connector	Pins chosen on the ESP32-S3's direct-routed (IO_MUX) pins; strapping pins kept off the headers
R7	Read the battery voltage in firmware	High-value resistor divider to an ADC1 pin (ADC1 still works while Wi-Fi is active)
R8	Easy and cheap to manufacture	2-layer, 1.6 mm, standard 6/6 mil rules, all parts on one side

2 Block diagram

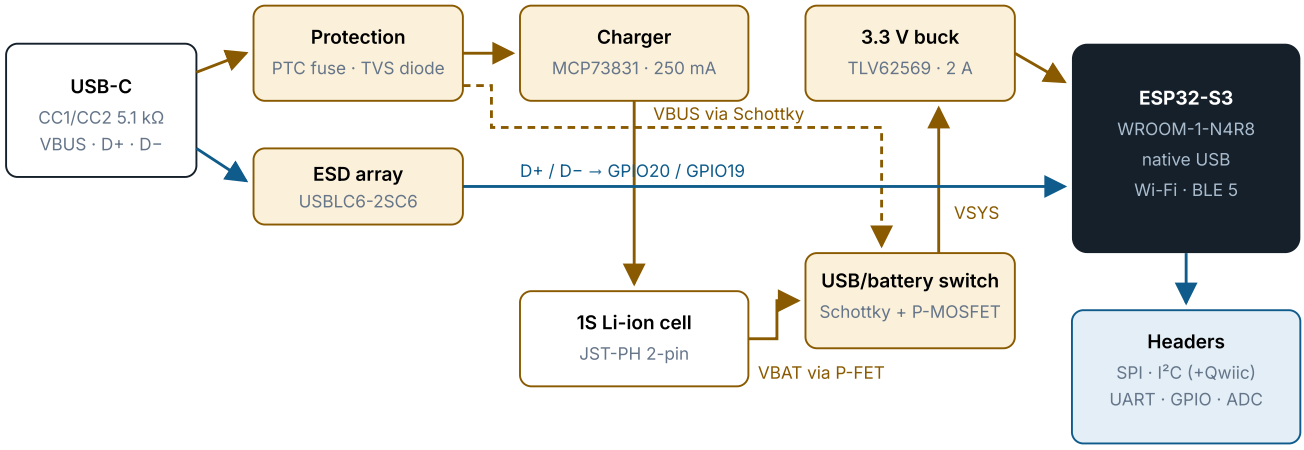


Figure 1. Block diagram. Amber: power. Blue: data and signals. When USB is connected, VBUS powers the board and charges the cell. When it is unplugged, the P-MOSFET switches the battery onto the supply rail.

3 Component selection and reasoning

Function	Part	Why this part
Microcontroller	ESP32-S3-WROOM-1-N4R8	Pre-certified radio module, so no RF design or approvals work. Native USB OTG and USB-Serial/JTAG remove the USB-to-serial chip and auto-reset transistors. 8 MB PSRAM for display and audio work. The same chip as my product firmware, so the drivers carry over.
USB-C connector	16-pin USB 2.0 Type-C receptacle (mid-mount or top-mount)	USB 2.0 only needs D+/D−, VBUS, GND, CC1 and CC2; the 24-pin version adds unused high-speed pairs and is harder to assemble. Both D+/D− pairs on the connector are joined so the cable works either way up.
CC termination	2 × 5.1 kΩ (1%) pull-down, CC1 and CC2	Tells a USB-C power source "this is a power sink", so it turns VBUS on. Without these resistors, a C-to-C cable from a phone charger gives 0 V. This is the most common USB-C dev-board mistake.
Data line ESD	USBLC6-2SC6	Low-capacitance ESD clamp made for USB 2.0 data lines, placed right next to the connector, before anything else on D+/D−.
VBUS surge	5 V unidirectional TVS diode (SMA/SOD-123)	Clamps hot-plug spikes and ESD on VBUS. Its breakdown voltage sits above the 5.25 V USB maximum.
Overcurrent	PTC resettable fuse, ~750 mA hold	Limits current from a short on the headers so it can't damage a laptop port; resets itself once the fault is removed.
Charger	MCP73831 (SOT-23-5)	Single-cell linear Li-ion charger: 4.20 V regulation, preconditioning of a deeply discharged cell, charge-status output pin. Charge current = 1000 V / R_{PROG}; **R_{PROG} = 4.02 kΩ → 249 mA** (Section 4 explains why 250 mA).
USB/battery switch	Schottky diode (VBUS) + P-MOSFET (battery), gate tied to VBUS with a 100 kΩ pull-down	When USB is plugged in, VBUS turns the P-FET off, so the system runs from USB through the diode and the charger only charges the cell. When USB is unplugged, the gate is pulled low, the FET turns on, and the battery feeds the rail through a low-resistance FET instead of a lossy diode.
3.3 V regulator	TLV62569 synchronous buck, 2.2 µH inductor	Input 2.5–5.5 V covers both USB and battery. Rated 2 A, enough for Wi-Fi peaks. At low battery it switches to 100% duty mode, so the output simply follows the input instead of dropping out. Output set by a divider: V_{OUT} = 0.6 V·(1 + 150 k/33 k) ≈ 3.33 V.
Reset / boot	EN: 10 kΩ pull-up + 1 µF; BOOT button on GPIO0	The RC delay holds EN low until the supply is stable, as Espressif recommends. Holding BOOT during reset enters the ROM bootloader, a recovery route if the application breaks the USB stack.
Battery sense	2 × 220 kΩ divider + 100 nF to GPIO1 (ADC1)	Halves the 4.2 V maximum into the ADC's range. High resistance keeps the constant drain under 10 µA; the capacitor gives the ADC a low-impedance source to sample.
Indicators	Power LED, charge LED (from the charger's STAT pin), RGB LED (WS2812-type) on GPIO48	Show power, charging and firmware state without a serial console.

I²C pull-ups	2 × 4.7 kΩ to 3.3 V, with solder-jumper to remove	Correct for 100–400 kHz with short wires; the jumper removes them when a sensor board already has its own.

> **Why a buck regulator instead of a linear regulator (LDO):** a 3.3 V LDO running from 5 V USB at a 300 mA average Wi-Fi load dissipates (5.0 − 3.3) × 0.3 ≈ **0.51 W**. In a SOT-23 package, with a typical θ_{JA} of about 200–250 °C/W, that is a **100 °C or more** temperature rise, enough to trip thermal shutdown. A buck regulator at about 90% efficiency loses around 0.11 W instead. The buck regulator's 100% duty mode also gives the low-battery behaviour an LDO would, so nothing is lost by using it.

4 Power budget

Current figures are planning estimates taken from typical datasheet values. Section 7 replaces them with measurements during bring-up.

Load on the 3.3 V rail	Typical	Peak	Notes
ESP32-S3 module, Wi-Fi transmitting	~ 120 mA average	~ 350 mA	Short peaks during transmit bursts; these set the bulk capacitance and regulator rating
ESP32-S3, BLE only / CPU active	~ 40–100 mA	~ 150 mA	Depends on clock speed and radio activity
RGB LED	~ 5 mA	~ 60 mA	Full white is worst case; firmware limits brightness
Power + charge LEDs	~ 4 mA	~ 4 mA	2 mA each
Header allowance (sensors, displays)	—	200 mA	Documented limit for users
Total on 3.3 V	**~ 130–230 mA**	**~ 615 mA**	2 A regulator → about 3× margin over the peak

Input current from USB

- **Converting the 3.3 V peak to 5 V input:** 0.615 A × 3.3 V / (0.9 × 5 V) ≈ **450 mA**.
- **Adding the charge current:** 450 + 250 ≈ **700 mA** worst case, when everything peaks while the battery is charging.
- **What a source guarantees:** USB-C sources advertise 1.5 A or 3 A on the CC lines. A plain USB 2.0 laptop port only guarantees **500 mA**. The design deals with that difference in three ways:
  1. The charge current is limited to 250 mA rather than the cell's maximum (0.5C on a 1000 mAh cell), keeping the typical draw within 500 mA.
  2. The worst-case peak only happens when Wi-Fi transmits while header loads are at full current. It is short and averages far lower.
  3. The CC voltage can optionally be read on an ADC pin to detect what the source advertises: about 0.4 V means default USB power, about 0.9 V means 1.5 A, about 1.7 V means 3 A. Firmware could then raise the charge current or limit the load.

Battery runtime (1000 mAh cell, planning estimate)

Usage profile	Average current	Estimated runtime
Wi-Fi always connected, sending regularly	~ 130 mA	~ 7 h
BLE advertising, CPU mostly idle	~ 30 mA	~ 30 h
Deep sleep, waking every minute	< 1 mA	weeks (limited by the divider, LEDs and regulator quiescent current)

Charge time at 250 mA is roughly 4.5–5 h from empty, including the constant-voltage phase at the end. That is acceptable for a development board. If the CC-detection option shows a 1.5 A source, firmware can allow a higher charge current.

Bulk and decoupling capacitance

- **ESP32-S3 module supply pin:** 22 µF bulk + 10 µF + 0.1 µF, as close to the pin as possible, so the Wi-Fi transmit current spike comes from the capacitors rather than through the regulator's response time.
- **Buck regulator:** 10 µF on the input, 22 µF on the output (X5R/X7R, rated ≥ 10 V).
- **Effective capacitance:** ceramic capacitors lose much of their capacitance under DC voltage. A 10 µF 0603 part at 5 V can measure about 40% lower, so the 0402 size is avoided for bulk capacitors.
- **Charger:** 4.7 µF on its input and on VBAT.

5 Pin plan

Pins were chosen to follow the ESP32-S3's hardware constraints, so the board doesn't fight the chip.

Interface	Signal → GPIO	Why these pins
USB	D− 19 · D+ 20	Fixed: the only native USB pins
SPI (FSPI)	CS 10 · MOSI 11 · SCK 12 · MISO 13	Direct-routed (IO_MUX) pins for SPI2: full 80 MHz clock without delays through the GPIO matrix
I²C	SDA 8 · SCL 9	Any pin can be I²C; these are free, next to each other, and also go to the Qwiic connector
UART1	TX 17 · RX 18	A spare UART for modems or GPS; UART0 (43/44) stays on its own 3-pin header for logs
Battery ADC	GPIO1 (ADC1_CH0)	ADC1 keeps working while Wi-Fi is active; ADC2 is shared with the Wi-Fi driver
CC sense (optional)	GPIO2 (ADC1_CH1)	Reads the CC voltage to detect source current (Section 4)
RGB LED	GPIO48	Free on the R8 module variant; the same pin as Espressif's DevKitC-1, so example code works unchanged
JTAG (optional header)	MTCK 39 · MTDO 40 · MTDI 41 · MTMS 42	External JTAG debugging, normally unnecessary because the built-in USB-JTAG covers it
General GPIO	4–7, 14–16, 21, 38, 47	Free and not strapping pins

PINS KEPT OFF THE HEADERS

- **GPIO26–37:** taken by the module's flash and octal PSRAM. Driving them crashes the chip.
- **Strapping pins GPIO0, 3, 45, 46:** read at reset to set the boot mode and flash voltage. GPIO0 only goes to the BOOT button. The others are left unconnected or held at their default with a resistor, so a header device can't stop the board booting.

CONNECTOR SET

- 2 × 2.54 mm side headers (breadboard-friendly)
- Qwiic / STEMMA-QT (JST-SH 4-pin) for I²C
- 3-pin UART0 console header
- JST-PH 2-pin battery connector, with polarity marked on the silkscreen
- Optional 2 × 5 1.27 mm JTAG header

6 PCB design rules

STACK-UP AND MANUFACTURING

- 2 layers, 1.6 mm FR-4, 1 oz copper, about 55 × 28 mm.
- Rules: 6 mil track/space, 0.3 mm vias, all parts on the top side (one assembly pass).
- Bottom layer: solid ground, broken only by short jumps of a trace where unavoidable.
- Mounting holes connected to ground, test points on VBUS, VBAT, VSYS, 3V3 and GND.

ANTENNA

- The module's PCB antenna hangs over the board edge, with no copper on any layer underneath it, as the module datasheet requires.
- No metal parts, headers or battery connector within about 15 mm of the antenna.

BUCK REGULATOR LAYOUT

- Input capacitor, IC and output capacitor in a small, tight loop. That high-current switching loop is the main source of noise.
- Small copper area on the switch node (to reduce radiated noise); the feedback trace routed away from the inductor.
- Ground connections close to the IC, with several vias down to the ground plane.

USB AND PROTECTION

- D+/D− routed as a pair with about 90 Ω differential impedance, short and of matching length, no vias, over unbroken ground.
- Optional 22 Ω series resistor footprints (fitted with 0 Ω at first).
- Order from the connector: **connector** → **ESD/TVS** → **fuse** → **everything else**, so a surge reaches the protection first.

Schematic sheets (KiCad, hierarchical)

Sheet	Contents
1 · USB input	USB-C connector, CC resistors, USBLC6, TVS, PTC fuse, optional CC-sense divider
2 · Power	MCP73831 + R_{PROG} + STAT LED, battery connector, USB/battery switch, TLV62569 + inductor + feedback divider, test points
3 · MCU	ESP32-S3-WROOM-1, decoupling, EN RC + reset button, BOOT button, strapping resistors, RGB LED
4 · I/O	Headers, Qwiic connector, I²C pull-ups with solder jumper, UART0 header, JTAG header, battery-sense divider

Design reviews before ordering: electrical rule check (ERC) with no errors, a footprint check against each datasheet's recommended land pattern, design rule check (DRC) with the manufacturer's rules, a 3D check of connector height and overhang, and a final check through every net on the USB-C connector pins.

7 Bring-up plan

The first boards are brought up one step at a time, powering each new section only after the previous one passes, with the bench supply current-limited so a mistake doesn't destroy the board. Every step has a pass criterion, which becomes the test log.

#	Step	Method	Pass criterion
1	Inspection, unpowered	Check under a microscope: component orientation, solder bridges at the USB-C connector and the regulator. Measure resistance VBUS→GND, VSYS→GND, 3V3→GND.	No bridges; every rail well above 1 kΩ to GND (no short)
2	Input stage	Bench supply 5.0 V, current limit 100 mA, into the VBUS test point. Then a USB-C charger through a C-to-C cable.	Idle current < 20 mA; VBUS present with the C-to-C cable (proves the CC resistors are right)
3	3.3 V rail	Measure 3V3 at no load and with a 500 mA electronic load; check ripple and switching on the scope.	3.30 V ± 3%; ripple < 30 mV peak-to-peak; stable switching
4	MCU alive	Connect by USB; `esptool.py chip_id` and `flash_id`.	USB-Serial/JTAG appears as a device; chip and flash read back correctly
5	Wi-Fi stress	Flash a continuous Wi-Fi transmit test; watch 3V3 on the scope at the module pin, triggering on the drop.	Lowest point > 3.0 V; no brownout resets over 10 minutes
6	Battery switch-over	Plug and unplug USB with a battery fitted, while running firmware.	No reset during the switch-over; VSYS goes from about VBUS − 0.3 V to about VBAT
7	Charging	Discharged cell plus an ammeter in series; log voltage and current until charging ends.	Constant current ≈ 250 mA ± 10%, then constant voltage at 4.20 V ± 1%; STAT LED turns off at the end
8	Peripherals	Bring-up firmware tests (Section 8).	All interface tests pass

Throughout, check every component's temperature with a thermal camera or thermocouple: the regulator, charger and P-FET should stay under about 60 °C at full load and full charge current.

8 Bring-up firmware

One ESP-IDF test application, controlled from a text menu over the built-in USB-Serial/JTAG console, so the whole board can be tested with one cable and no extra equipment.

Test command	What it does	Pass criterion
`gpio`	Steps through the header pins one at a time (connect them to a logic analyser, or loop them back in pairs)	Each pin toggles; no two pins are shorted together
`uart`	UART1 loopback with TX 17 wired to RX 18, at 115200 and 921600 baud	10,000 random bytes sent and received correctly at both speeds
`spi`	SPI2 loopback with MOSI wired to MISO, at 1, 10, 40 and 80 MHz	4 KB transferred by DMA and read back identical at every clock rate
`i2c`	Scans addresses 0x08–0x77 with a known sensor on the Qwiic connector	The sensor is found at its address and its ID register reads correctly
`adc`	Reads the battery divider, corrected with the ADC's factory calibration	Within ±2% of a multimeter reading at VBAT
`led`	Cycles the RGB LED through red, green, blue and white	All colours correct
`wifi`	Connects to an access point, measures signal strength and pings in a loop	Connects; no brownout or reset in 10 minutes
`sleep`	Deep sleep with a timer wake-up, measuring the whole board's current	Matches the deep-sleep estimate in Section 4 (anything much higher means something is leaking current)

The structure of this firmware comes from my product work: drivers are separated from the test logic, every failure gets a specific error code, and results are printed in a form that can be pasted straight into the test log.

9 Risks and next steps

Risk	Mitigation
Wrong USB-C footprint or pin mapping	Check the footprint against the exact connector's datasheet; print it at 1:1 scale and lay the real part on the paper; continuity test before soldering the module
Supply droop during Wi-Fi peaks	Bulk capacitors at the module, a 2 A regulator, and step 5 of the bring-up plan
Loads above 500 mA on a laptop USB port	250 mA charge current, optional CC detection, documented header current limit
Poor Wi-Fi range because of the layout	Copper-free area under the antenna; signal strength checked in bring-up
Battery wired backwards	Polarised JST-PH connector, polarity marked on the silkscreen; reverse-polarity FET as a later improvement

Status and next steps

Stage	Status
Requirements, architecture, component choices, power budget, pin plan	Done (this document)
Schematic capture in KiCad, plus ERC	Next
PCB layout, plus DRC and design review	Planned
Fabrication and assembly	Planned
Bring-up and test log (Sections 7–8)	Planned

The current figures, efficiency and component values are planning estimates from typical datasheet figures. They must be checked against each part's current datasheet before ordering, and replaced with measurements during bring-up.